



An inter- and intra-attention model for multi-behavior recommendation

Shereen Elsayed¹ · Ahmed Rashed² · Lars Schmidt-Thieme¹

Received: 27 November 2024 / Accepted: 29 May 2025 / Published online: 2 July 2025
© The Author(s) 2025

Abstract

In the context of recommendation systems, addressing the multi-behavior recommendation problem has become increasingly vital for online platforms seeking to comprehend users' dynamic behavior over time. Recent models employ various techniques, such as graph neural networks and attention mechanisms, to learn users' behaviors. However, similar to the sequential recommendation, preserving sequential dynamic patterns in multi-behavior recommendation remains crucial. Existing models encounter challenges in learning deep behavioral representations within a given sequence while simultaneously maintaining the temporal order of items. To address this issue, we introduce a novel inter- and intra-attention mechanism for multi-behavior recommendation (IIARec). Specifically, our approach applies intra-self-attention to items of the same behavior, followed by inter-self-attention across all behaviors. Additionally, we propose historical behavior indicators to encode the historical frequency of each item's behavior in the input sequence. Furthermore, the IIARec model operates in a multitask setting, allowing it to learn item behaviors and their associated scores concurrently. Extensive experimental results on four real-world datasets demonstrate that our proposed model outperforms state-of-the-art methods, illustrating its efficiency in addressing the multi-behavior recommendation challenge.

Keywords Sequential recommendation · Multi-behavior recommendation · Multitask learning

1 Introduction

Recommender systems have become essential on modern online platforms, especially with the massive increase of number of users. Their primary goal is to enhance user satisfaction by helping them discover preferred items while simultaneously boosting platform revenue. Recommender systems approaches have evolved, from traditional matrix factorization to modern neural network-based methods like neural collaborative filtering (NCF) [6]. Recommender sys-

tems are also categorized into specialized areas, including sequential recommendations, which considered the historical order of the user's interacted items such as SASRec [9] and BERT4Rec [14]. Furthermore, context- and attribute-aware recommendations that consider users and items attribute into account such as CARCA [13], SSE-PT [24] and TiSASRec [12].

On many platforms, users interact with items through diverse behaviors beyond purchases, such as viewing items or adding them to a favorites list. These interactions provide valuable insights into user preferences and intentions. However, traditional sequential recommendation techniques often focus solely on primary user-item relationships, typically emphasizing purchase interactions in retail settings. This approach overlooks the richness of other interaction data, limiting the potential for more refined recommendations.

To address this limitation, recent advancements in multi-behavior recommendation aim to integrate all user interactions (e.g., clicks, purchases, and add-to-cart actions) into the next-item recommendation task. By leveraging the full spectrum of user behaviors, these approaches yield more

✉ Shereen Elsayed
elsayed@ismll.uni-hildesheim.de

Ahmed Rashed
ahmed.rashed@vwfs.io

Lars Schmidt-Thieme
schmidt-thieme@ismll.uni-hildesheim.de

¹ Information Systems and Machine Learning Lab (ISMLL)& VWFS Data Analytics Research Center (VWFS DARC), University of Hildesheim, Hildesheim, Germany

² Data Analytics & AI Engineering, Volkswagen Financial Services AG, Braunschweig, Germany

insightful and personalized recommendations. For instance, on retail platforms, actions like clicking on an item or adding it to a favorites list can signal a strong intent to purchase. Graph-based methods, such as MB-GCN [8] and MB-GMN [19], construct user–item and item–item graphs to model diverse behaviors. More recent approaches, like MB-STR [23] and MBHT [21], utilize attention mechanisms to capture behavioral patterns. However, these methods often struggle to fully represent complex behavioral relationships and sequential dependencies in user interaction histories.

To bridge this gap, we propose IIARec, a novel framework that employs an inter–intra-attention mechanism to model deep relationships both within the same behavior and across different behaviors. Capturing these intricate dependencies while preserving the temporal dynamics of the input sequence is a significant challenge. To address this, we introduce item masking, which selectively masks items not belonging to the same behavior, enabling the model to focus on relevant behavioral dependencies while maintaining the chronological order of interactions. Furthermore, we incorporate historical behavior indicators as contextual information, which track the frequency of each item’s behavior in the input sequence as illustrated in Fig. 1.

Our contributions can be summarized as follows:

- We present the IIARec model for the multi-behavior recommendation, incorporating inter–intra-attention to learn comprehensive behavioral representations. Furthermore, we employ multitask learning to understand both user intentions and interaction types.
- Within the IIARec model, we introduce hierarchical masked attention that models deep behavioral relationships within and across behaviors while preserving input sequence dynamics. We also include historical behavior indicators (HBI) to tackle item behavior frequency.
- Extensive experiments on four real-world datasets show that our proposed model IIARec outperforms previous state-of-the-art multi-behavioral recommendation models.

2 Related work

The significance of recommendation systems has witnessed remarkable growth in recent years, significantly impacting the revenue and online advertising strategies of online platforms. In the initial stages, recommendation systems leaned on factorization machines. However, as time progressed, more advanced techniques emerged, making use of neural networks such as neural collaborative filtering (NCF) [6] and neural factorization machine (NFM) [5] to capture the nonlinearities inherent in user–item interactions. The related work section can be broadly classified into two primary

categories: sequential recommendation and multi-behavioral recommendation.

Sequential recommendation is a pivotal task within recommender systems, leveraging users’ historical interactions to predict the items they are most likely to interact with. In its early stages, approaches hinged on convolutional neural networks (CNNs) [15] and gated recurrent units (GRUs) [7] for encoding user interaction histories. With the proven success of transformer architectures in diverse domains like natural language processing and image analysis, recent sequential recommendation models have adopted transformer structures to harness the power of attention mechanisms for capturing sequential correlations within input sequences. This form of recommendation has gained paramount importance across various online platforms. Notably, SASRec [9] was among the pioneering models to incorporate the transformer architecture in sequential recommendation. Subsequent models, such as BERT4Rec [14], have further refined this approach by employing bidirectional self-attention to model intricate sequential patterns. Furthermore, innovations in this domain include SSE-PT [24] and TiSASRec [12]. On another front, contemporary models like CARCA [13] incorporate contextual information and item attributes, where the sequence of items and the target items pass through a cross-attention mechanism to get the final items’ scores. Recent approaches have also turned to contrastive techniques to enhance model performance, as seen in ICLRec [2]. In recent times, recommendation models have increasingly turned to large language models. For instance, models like RecFormer [11] leverage the LongFormer model to encode item sequences based on the textual attributes associated with those items.

With the current massive data sources available for recommender systems, recent models started not only to employ the user–item primary interaction relation such as purchases but also to use other available click data. Employing auxiliary information has different namings in the existing prior work: relation-aware recommender systems, multi-relation recommendation, and multi-behavior recommendation.

Multi-behavior Recommendation has garnered significant attention in recent years, driven by the proliferation of diverse data sources in online applications, resulting in a remarkable boost in recommendation precision. Graph-based methodologies have demonstrated effectiveness in modeling user–item and item–item relations; MB-GCN [8] utilizes a graph convolutional network for modeling the user–item multi-behavior relations. On the other hand, MB-GMN [19] employs a meta-learning paradigm to learn behavior heterogeneity. More recent approaches like KHGT [18] use graph attention layer to model the item–item and user–item relation. Recognizing the temporal dimension in multi-behavior recommendation and its connection to sequential recommendation has given rise to a new category of models, including MBHT [21] and MB-STR [23] that employ atten-

tion mechanism while considering the sequential pattern in the input behaviors. As an advancement beyond graph-based approaches, cascaded graph convolutional networks [3] were recently proposed to learn behavior dependencies in embedding learning. Knowledge enhancement for contrastive multi-behavior recommendation (KMCLR) [20] incorporates contrastive learning into the multi-behavior recommendation paradigm to address data sparsity issues in input data. Another contrastive learning approach is the CLMRS framework [22], which employs a meta-network to encode and capture the semantic information present in multi-behavior data. Additionally, the efficient noise-decoupling model (END4Rec) [4] enhances the understanding of user behavior sequences by incorporating item information, behavior information, and location information, enabling a more comprehensive representation of user interactions.

In contrast to the previously mentioned models, our approach integrates the strengths of both model families. It captures sequential patterns in the data using a transformer-based encoder while simultaneously leveraging auxiliary information to enhance user preference understanding. By combining these advantages, our model delivers superior recommendation performance (Table 1).

3 Methodology

In this section, we introduce the IIARec model, which incorporates a historical behavior indicator as contextual information, representing the frequency of each action on a specific item. We also detail the inter- and intra-attention mechanisms designed to capture complex, multi-behavioral sequential patterns. Finally, we describe the multitask learning framework of IIARec, where the model simultaneously learns item scores and user behaviors through an auxiliary classification task.

3.1 Problem formulation

In multi-behavior settings, given a set of items $\mathcal{I} := \{1, \dots, I\}$ and a set of users $\mathcal{U} := \{1, \dots, U\}$, each user u has a historical ordered sequence S of item interactions $S^u := \{v_1^u, \dots, v_{|S^u|}^u\}$. Given K behaviors, each item in the sequence has a corresponding behavior type $b \in \mathcal{B} := \{1, \dots, K\}$, such as buy and add-to-favorite. Our goal is to predict the next item to be interacted with based on the primary behavior, while other behaviors serve as auxiliary information to better model the user behavior over time.

3.2 IIARec

Figure 2 illustrates the model architecture. It consists of item and behavior encoding, hierarchical masked attention, and multitask learning.

3.2.1 Item and behavior encoding

As discussed earlier, each user has an input sequence of historical item interactions S^u and a corresponding behavior sequence $B^u = [b_1, b_2, \dots, b_{|S^u|}]$. To construct the latent item representation, we first embed each item in the input sequence using a fully connected layer to obtain the initial embeddings as:

$$v'_j = v_j W_v + B_v \quad (1)$$

where $W_v \in \mathbb{R}^{I \times d}$ is the weight matrix, I is the number of items, and d is the item embedding dimension and B_v is the bias term. To encode item behaviors, we use historical behavior indicators c_j that track the frequency of each behavior type on an item throughout the sequence. Figure 1 shows an example of the historical behavior indicators in a sequence. We also embed these indicators using a fully connected layer. This allows the model to learn the order and the frequency of user behaviors on items.

$$c'_j = c_j W_c + B_c \quad (2)$$

where $W_c \in \mathbb{R}^{N \times d}$ is the weight matrix, N is the number of different historical behavior indicators, d is the context embedding dimension and B_c is the bias term. For simplicity, we use the **same dimension** d for all embedding layers.

Afterward, we concatenate both embeddings to get the combined item encoding r_j . Then, feed it to another fully connected layer for embedding item and historical behavior indicator together:

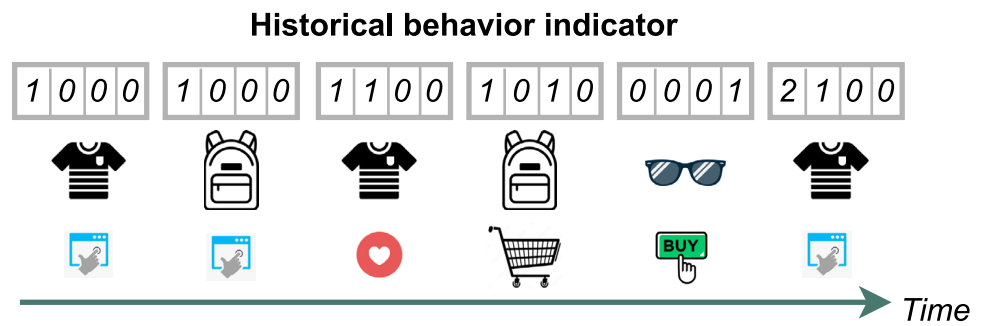
$$r_j = \text{concat}_{\text{col}}(v'_j, c'_j), \quad q_j = r_j W_r + B_r \quad (3)$$

where $W_r \in \mathbb{R}^{2d \times d}$ is the weight matrix, d is the layer embedding dimension, and B_r is the bias term. Finally, a learnable positional encoding is added to the final embedding to indicate the position in the input interactions sequence:

$$g_j = q_j + P_j \quad (4)$$

3.2.2 Inter-Intra-attention

In sequential multi-behavior recommendation, sequence order is crucial for an accurate next-item recommendation.

Fig. 1 Historical behavior indicator example**Table 1** Notation used in the paper

Symbol	Description
$\mathcal{U}$	Set of users in the dataset
$\mathcal{I}$	Set of items in the dataset
$\mathcal{B}$	Set of behaviors in the dataset (e.g., click, purchase)
S^u	User sequence of interactions
K	Total number of behaviors
M_b^u	Behavior-specific mask for user u and behavior type b
α_b	Behavior weight for item i in the sequence

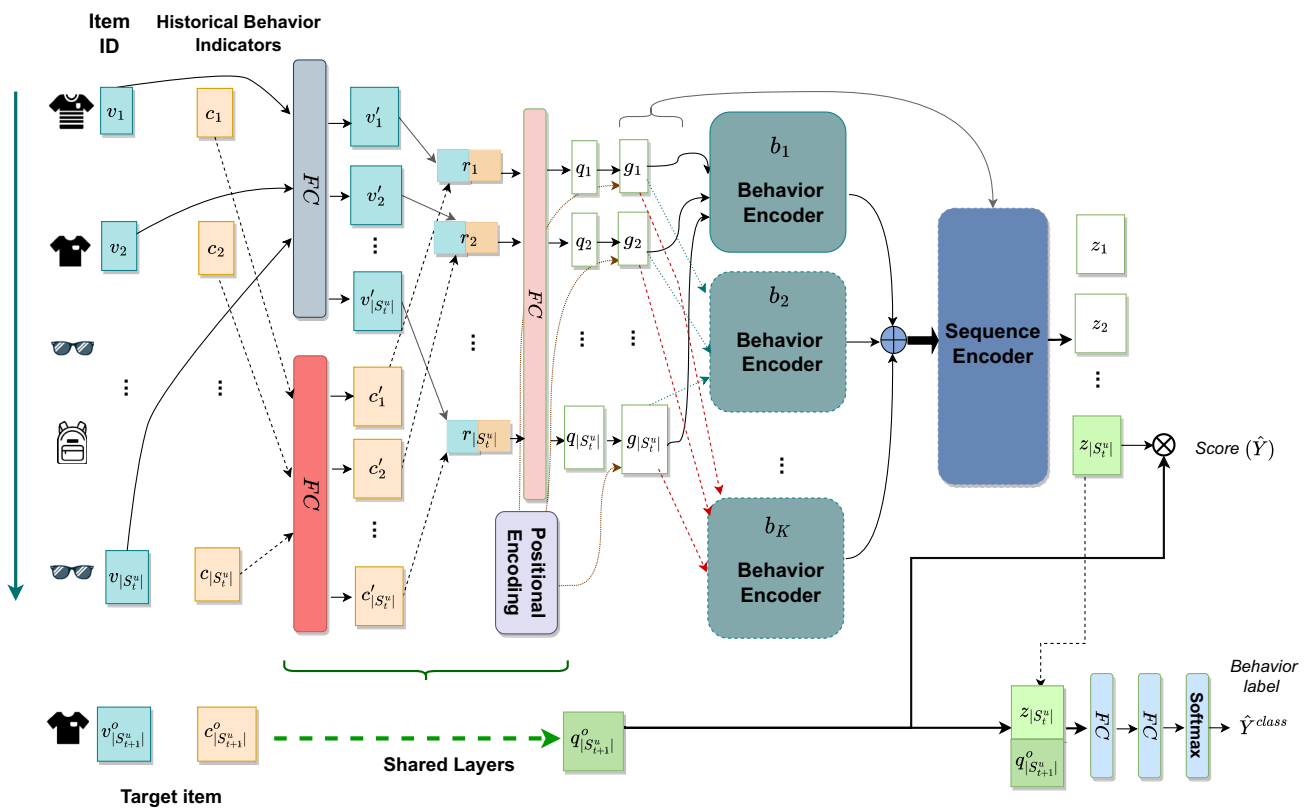
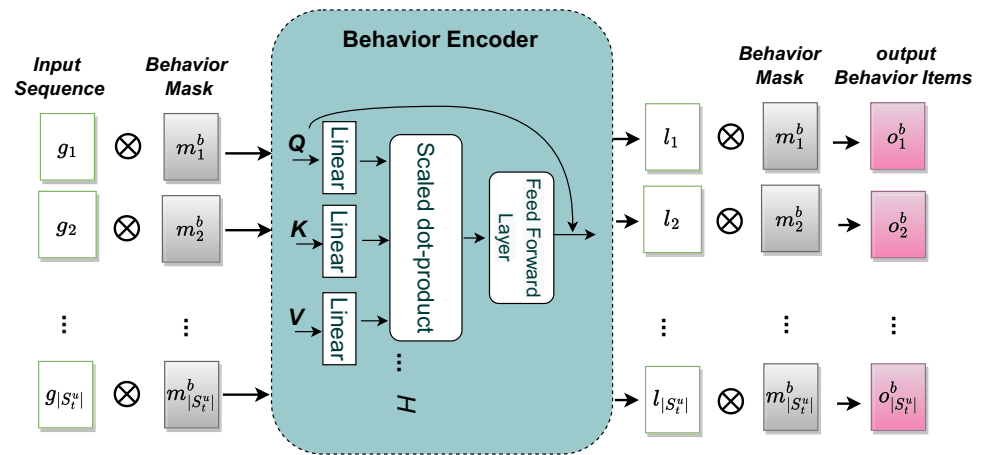
**Fig. 2** Inter- and intra-attention model architecture

Fig. 3 Behavior encoder architecture

In our work, we model behavioral interactions using two-stage self-attention to preserve input sequence order. The first stage is the **behavior encoder**, which encodes the items with the same behavior in the sequence, followed by a **sequence encoder**, which encodes the whole sequence and allows the modeling of different behaviors in the input sequence.

3.2.3 Behavior encoder (intra-attention)

Recent approaches use graphs to gather multi-behavior information, potentially sacrificing the input sequence's sequential pattern. To maintain the items' order while obtaining better representations, we employ a behavior mask M_b^u , created from the sequence's items, to filter out items not associated with a specific behavior b . Accordingly, we multiply the item sequence embedding $G^u := [g_1, g_2, \dots, g_{|S_t^u|}]$ and with the corresponding mask of the specified behavior $M_b^u := [m_1^b, m_2^b, \dots, m_{|S_t^u|}^b]$ as shown in Fig. 3.

This yields a number of behavior encoder blocks equivalent to the number of behaviors K . The masked latent embeddings are then fed into a fully connected layer to get $E_b^u := \{e_1^b, \dots, e_{|S_t^u|}^b\}$ followed by a multi-head self-attention component to sequentially encode items with the same behavior as follows:

$$e_j^b = (g_j * m_j^b) W_e + B_e, \quad W_e \in \mathbb{R}^{d \times d}, \quad B_e \in \mathbb{R}^d \quad (5)$$

$$X_b^u = \text{SA}(E_b^u) = \text{concat}_{\text{col}} \left(\text{Att}(E_b^u \mathbf{W}_h^{Q^b}, E_b^u \mathbf{W}_h^{K^b}, E_b^u \mathbf{W}_h^{V^b}) \right)_{h=1:H} \quad (6)$$

where $\mathbf{W}_h^{Q^b}, \mathbf{W}_h^{K^b}, \mathbf{W}_h^{V^b} \in \mathbb{R}^{d \times \frac{d}{H}}$ represent the linear projection matrices of the head at index h , and H is the number of heads. X_b^u represents the column-wise concatenation of the attention heads. Finally, we have the point-wise feed-forward layers to obtain the component's final output representations

$F_b^u \in \mathbb{R}^{|S_t^u| \times d}$, $F_b^u = \text{FFN}(X_b^u)$ as follows:

$$\text{FFN}(X_b^u) = \text{concat}_{\text{row}} \left(\text{ReLU}(X_{b,j}^u \mathbf{W}^{(1)b} + \mathbf{B}^{(1)b}) \mathbf{W}^{(2)b} + \mathbf{B}^{(2)b} \right)_{j=1:|S_t^u|} \quad (7)$$

where $\mathbf{W}^{(1)b}, \mathbf{W}^{(2)b} \in \mathbb{R}^{d \times d}$ are the weight matrices of the two feed-forward layers, and $\mathbf{B}^{(1)b}, \mathbf{B}^{(2)b} \in \mathbb{R}^d$ are their bias vectors. The initial embedding sequence is added via a ReZero [1] residual connection to the behavior encoder output to obtain as:

$$L_b^u = F_b^u + \gamma (G^u * M_b^u) \quad (8)$$

where γ is a learnable weight initialized with zero to adjust the contribution of $(G^u * M_b^u)$. The output is multiplied by the behavior mask again to mask the positions of the other behaviors as follows:

$$O_b^u = L_b^u * M_b^u \quad (9)$$

Finally, we combine the output sequences of each behavior $O_b^u := [o_1^b, o_2^b, \dots, o_{|S_t^u|}^b]$ by summing them element-wise to generate the first stage multi-behavioral latent sequence embeddings O^u as follows:

$$O^u = \sum_{b=0}^K O_b^u \quad (10)$$

3.2.4 Sequence encoder (inter-attention)

In the second phase of inter-intra-attention, we utilize a multi-head self-attention component across all behaviors while maintaining the original temporal order of the sequence. Specifically, once we have constructed the sequence comprising all behaviors, denoted as O^u , the

sequence encoder employs attention mechanisms over all items within the sequence. This enables the model to capture dependencies among various behaviors within the sequence, yielding an inter- and intra-multi-behavioral latent representation for each item:

$$A^u = \text{SA}(\mathcal{O}^u) = \text{concat}_{\text{col}} \times \left(\text{Att}(\mathcal{O}^u \mathbf{W}_h^Q, \mathcal{O}^u \mathbf{W}_h^K, \mathcal{O}^u \mathbf{W}_h^V) \right)_{h=1:H} \quad (11)$$

where $\mathbf{W}_h^Q, \mathbf{W}_h^K, \mathbf{W}_h^V \in \mathbb{R}^{d \times \frac{d}{H}}$ represent the linear projection matrices of the head at index h , and H is the number of heads. A^u represents the column-wise concatenation of the attention heads. Additionally, for the model stability, we add a residual connection between sequence attention output A^u and sequence items G^u :

$$A^{u'} = A^u + G^u \quad (12)$$

Finally, we have the point-wise feed-forward layers to obtain the component's output representations $Z^u \in \mathbb{R}^{|S_t^u| \times d}$ as follows:

$$Z^u = \text{FFN}(A^{u'}) = \text{concat}_{\text{row}} \times \left(\text{ReLU}(A^{u'} \mathbf{W}^{(1)Z} + \mathbf{B}^{(1)Z}) \mathbf{W}^{(2)Z} + \mathbf{B}^{(2)Z} \right)_{j=1:|S_t^u|} \quad (13)$$

where $\mathbf{W}^{(1)Z}, \mathbf{W}^{(2)Z} \in \mathbb{R}^{d \times d}$ are the weight matrices of the two feed-forward layers, and $\mathbf{B}^{(1)Z}, \mathbf{B}^{(2)Z} \in \mathbb{R}^d$ are their bias vectors.

3.3 Multitask learning

In order to train the model, we utilize a multitask learning approach to learn item ranking and behavior type simultaneously. For efficient training, we adopt the SASRec [9] process, which uses autoregressive training, considering only historical items for each target item recommendation. The input sequence, S^u , has a fixed length, achieved by padding or truncating. The target sequence is constructed by right-shifting the input sequence and including the last item interaction. On the other hand, negative items are sampled using random items not in the user's interaction history $v \notin S^u$.

For item ranking, we calculate the final score by taking the dot product of the last item embedding $z_{|S_t^u|}$ from the sequence encoder and the target item embedding $q_{|S_{t+1}^u|}^o$ as follows:

$$\hat{Y}_{t+1} = \sigma(z_{|S_t^u|} \cdot q_{|S_{t+1}^u|}^o) \quad (14)$$

where σ is a sigmoid function. In multi-behavior datasets, interactions have varying importance for next-item recommendation. To handle this, we introduce weighting factors α_b in the loss function, one for each behavior. The number and values of α_b may differ based on the dataset's behavior count and their importance. Thus, our weighted binary cross-entropy objective for multi-behavior recommendation is as follows:

$$\mathcal{L}_{\text{rank}} = - \sum_{S^u \in \mathcal{S}} \sum_{t=0}^{|S^u|} [\alpha_b \log(\hat{Y}_t^{O^{(+)}}) + \beta (\log(1 - \hat{Y}_t^{O^{(-)}}))] \quad (15)$$

where $\hat{Y}_t^{O^{(+)}}$ are the output scores for the positive samples and $\hat{Y}_t^{O^{(-)}}$ are the output scores for the negative samples, $\mathcal{S}$ is the set of all sequences, α_b is the behavior weights, and lastly, β is the sampling weight.

In this work, we also propose learning target interaction behavior type as an auxiliary task enhancing the model's ability to understand user behavior patterns and anticipate the behavior type of the next interaction. The behavior-type task involves a two-layer neural network that takes concatenated item embeddings $z_{|S_t^u|}$ and $q_{|S_{t+1}^u|}^o$ as input, as follows:

$$\hat{Y}^{\text{class}} = \text{Softmax}((\text{concat}_{\text{col}}(z_{|S_t^u|}, q_{|S_{t+1}^u|}^o) \mathbf{W}^{(1)} + \mathbf{B}^{(1)}) \mathbf{W}^{(2)} + \mathbf{B}^{(2)}) \quad (16)$$

where $\mathbf{W}^{(1)} \in \mathbb{R}^{2d \times d}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{d \times d}$ are the weight matrices of the two fully connected layers, and $\mathbf{B}^{(1)}, \mathbf{B}^{(2)} \in \mathbb{R}^d$ are their bias vectors. Then, we apply the softmax function on the output to get the label scores of each behavior and utilize the cross-entropy classification loss as follows:

$$\mathcal{L}_{\text{class}} = - \sum_{S^u \in \mathcal{S}} \sum_{t=0}^{|S^u|} \sum_{j=0}^K [Y_{t_j}^{\text{class}} \log(\hat{Y}_{t_j}^{\text{class}})] \quad (17)$$

where Y_{t_j} is the true behavior labels and $\hat{Y}_{t_j}^{\text{class}}$ is the predicted classification scores. Finally, the model is optimized using ADAM optimizer [10]. The final model loss can be defined as a weighted sum of the two losses:

$$\mathcal{L}_{\text{model}} = \mathcal{L}_{\text{rank}} + \theta \mathcal{L}_{\text{class}} \quad (18)$$

where θ is a hyper-parameter controlling the contribution of the classification task to the final loss.

4 Experiments

In this section, we aim to address the following research questions:

Table 2 Dataset statistics

Dataset	Interactions	User#	Item#	Behaviors
Taobao	7,658,926	147,894	99,037	Page View, Fav., Cart, Buy
Yelp	1,400,000	19,800	22,734	Tip, Dislike, Neutral, Like
MovieLens	9,922,036	67,788	8704	Dislike, Neutral, Like
Tianchi	4,619,389	25,000	500,900	Page View, Fav., Cart, Buy

- **RQ1:** How does the IIARec model’s performance compare to that of state-of-the-art multi-behavior recommendation models?
- **RQ2:** What influence does each individual model component have on the overall model performance?
- **RQ3:** What impact do auxiliary behaviors have on the model’s performance?
- **RQ4:** What is the model sensitivity against hyper-parameter tuning?

4.1 Experimental settings

4.1.1 Datasets

We assess our model on four multi-behavioral datasets:

Taobao¹ [18, 23] : Data from the popular Chinese online shopping platform Taobao with behaviors like buy, add-to-cart, add-to-favorite, and page view. We consider buy behavior as the target behavior. **Tianchi**² : Collected from Tmall.com, similar to Taobao, it includes buy, add-to-cart, add-to-favorite, and page view behaviors. **Yelp**³ [18, 23] : Data gathered from the Yelp challenge, categorizing behaviors based on user ratings. Behaviors are split into dislike (rating ≤ 2), natural ($2 < \text{rating} < 4$), and like (rating ≥ 4). Users can also write venue tips. **MovieLens**⁴ [18] : Similar to Yelp, but with movie ratings, categorizing behaviors as dislike, neutral, and like. The main behavior considered for both MovieLens and Yelp datasets is the “like” behavior. Dataset statistics are summarized in Table 2.

4.2 Evaluation protocol

For a fair comparison, we follow the same evaluation process as recent state-of-the-art methods [18] [23]. We use a leave-one-out mechanism, training and validating the model with the entire sequence except the last interaction, which is used for testing. We generate 99 negative items for each positive item and assess model performance with hit ratio (HR@N)

and normalized discounted cumulative gain (NDCG@N). Higher HR and NDCG values indicate better performance. We report the mean and standard deviation of results from three separate runs to ensure statistical robustness.

4.2.1 Baselines

We compare our proposed method against various sequential and multi-behavioral recommendation methods.

Sequential and Context-Aware Recommendation Methods.

- **SASRec** [9]: A model that utilizes multi-head self-attention to capture the sequential pattern in the users’ history and then applies dot product for calculating the item scores.
- **SSE-PT** [16]: A state-of-the-art model which incorporates the user embedding into a personalized transformer with stochastic shared embedding regularization for handling extremely long sequences.
- **CARCA** [13]: A context-aware sequential recommendation approach employing cross-attention between user profiles and items for score prediction.

Multi-behavior recommendation models.

- **MATN** [17]: A memory-augmented transformer network that utilizes a transformer-based encoder that jointly models behavioral dependencies.
- **MB-GCN** [8]: A model that represents the data as a unified graph, then employs a graph convolutional network to learn the node’s representation.
- **MB-GMN** [19]: Learns the multi-behavior dependencies through graph meta-network.
- **KHGT** [18]: This model employs a knowledge graph hierarchical transformer to capture the behavior dependencies in the recommender model.
- **MB-STR** [23]: Employs multi-behavior sequential transformers to learn sequential patterns across various behaviors.
- **MBHT** [21]: Utilizes a hyper-graph-enhanced transformer model with low-rank self-attention to model short and long behavioral dependencies.

¹ <https://github.com/akaxlh/MB-GMN/tree/main/Datasets/Tmall>.

² <https://tianchi.aliyun.com/competition/entrance/231576/information>.

³ <https://github.com/akaxlh/KHGT/tree/master/Datasets/Yelp>.

⁴ <https://github.com/akaxlh/KHGT/tree/master/Datasets/MultiInt-ML10M>.

Table 3 Model performance and comparison against baselines on Taobao and Yelp datasets

Method	Taobao HR@10	NDCG@10	Yelp HR@10	NDCG@10
<i>Sequential and context-aware recommendation methods</i>				
SASRec [9]	0.390 ± 4.2E-3	0.249 ± 8E-4	0.853 ± 1.7E-3	0.5601 ± 5.1E-3
SSE-PT [16]	0.393 ± 2.7E-3	0.232 ± 3E-4	0.857 ± 1.5E-2	0.572 ± 1.8E-2
CARCA [13]	0.769 ± 4E-3	0.662 ± 5E-3	0.854 ± 2E-3	0.589 ± 4E-3
<i>Multi-behavior recommendation methods</i>				
MATN [17]	0.354†	0.209†	0.826†	0.530†
MB-GCN [8]	0.369†	0.222†	0.796†	0.502†
MB-GMN [19]	0.491†	0.300†	0.861 ± 8E-3	0.570 ± 1.2E-2
KHGT [18]	0.464†	0.278†	0.880†	0.603†
KMCLR [20]	0.4557 †	0.2735 †	0.8897 †	0.6038 †
MB-STR [23]	0.768†	0.608†	0.882†	0.624†
MBHT [21]	0.745 ± 6.1E-3	0.559 ± 8.4E-3	0.885 ± 5E-4	0.618 ± 1.4E-3
IIARec (ours)	0.8515 ± 1.8E-3	0.7294 ± 1.2E-3	0.9015 ± 5.7E-5	0.6374 ± 8.08E-4
Improv.(%)	10.663%	10.120%	1.349%	2.083%

Published results are indicated by the † symbol. Bold represents the best results and Italic represented the second best

Table 4 Model performance and comparison against baselines on MovieLens and Tianchi datasets

Method	MovieLens HR@10	NDCG@10	Tianchi HR@10	NDCG@10
<i>Sequential and context-aware recommendation methods</i>				
SASRec [9]	0.911 ± 1E-3	0.668 ± 5.1E-3	0.659 ± 3E-3	0.495 ± 2E-3
SSE-PT [16]	0.911 ± 7.1E-3	0.657 ± 4.5E-3	0.663 ± 1.2E-2	0.468 ± 1.3E-2
CARCA [13]	0.906 ± 2E-3	0.665 ± 1E-3	0.713 ± 4E-4	0.500 ± 1E-3
<i>Multi-behavior recommendation methods</i>				
MATN [17]	0.847†	0.569†	0.714 ± 7E-4	0.485 ± 2E-3
MB-GCN [8]	0.826†	0.553†	-	-
MB-GMN [19]	0.820 ± 1.1E-3	0.530 ± 9E-4	0.737 ± 4.3E-3	0.502 ± 1.9E-3
KHGT [18]	0.861†	0.597†	0.652 ± 1E-4	0.443 ± 1E-4
MBHT [21]	0.913 ± 5.9E-3	0.695 ± 7E-3	0.725 ± 6.3E-3	0.554 ± 4.8E-3
IIARec (ours)	0.9412 ± 5.5E-4	0.7370 ± 1.7E-3	0.7842 ± 1.0E-3	0.5974 ± 1.7E-3
Improv.(%)	3.066%	6.043%	6.377%	7.761%

Published results are indicated by the † symbol. Bold represents the best results and Italic represented the second best

Table 5 Ablation study on model components (HR@10)

Model	Dataset Taobao	Yelp	Tianchi	MovieLens
w/o Multitask	0.846	0.897	0.781	0.943
w/o HBI	0.851	0.901	0.780	0.942
w/o Behavior encoder	0.840	0.893	0.779	0.936
IIARec	0.851	0.901	0.784	0.941

- **KMCLR [20]**: A knowledge graph-based approach that utilizes contrastive learning to capture the commonalities between different behaviors.

4.3 Model performance (RQ1)

We conducted experiments to evaluate our proposed model HMAR against several state-of-the-art approaches. Looking into Tables 3 and 4 SASRec and SSE-PT results, they show very high competitive results against the state-of-the-art multi-behavioral models. On the other hand, methods such as MATN, MB-GCN, MB-GMN, and KHGT show fragile results, specifically on the Taobao dataset. Nevertheless, our model IIARec significantly outperforms all types of baselines and improves the HR on the Taobao dataset and the Tianchi dataset. In this case, we compete against the state-of-the-art context-aware method CARCA, and our model IIARec shows significantly better results on the four datasets with

a reasonable margin. Regardless, the CARCA model shows competitive results as it benefits from both sequential and similar contextual information that exists in the data.

4.4 Individual model components (RQ2)

As shown in Table 5, particularly, the historical behavior indicator (HBI) significantly benefits the Tianchi dataset, given the influence of behaviors like page views and favorites on performance. However, there is no observable effect on the MovieLens and Yelp datasets. Multitask learning demonstrates its significance in datasets where auxiliary behaviors play a more substantial role but has a less pronounced impact on others, such as the MovieLens dataset. Additionally, eliminating the behavior encoder component, responsible for capturing deep behavioral dependencies, and relying solely on the sequence encoder, has an impact on all datasets, highlighting the advantages of the two-stage attention procedure for modeling multi-behavior sequences.

4.5 Effect of auxiliary behaviors (RQ3)

Figures 4 and 5 demonstrate the importance of auxiliary behaviors in enhancing the model's understanding of user behavior, particularly for the Tianchi and Taobao datasets. When only the target behavior interactions were used, and auxiliary behaviors were ignored, the performance was significantly impacted. For the Tianchi dataset, removing click interactions caused a substantial drop in HR, decreasing from 0.7842 to 0.6638. In contrast, the removal of “add-to-favorites” behavior had a negligible effect. Additionally, removing the “add-to-cart” behavior did not noticeably affect performance, likely because all test items were excluded from the user input sequence in this setup. This exclusion was designed to avoid making the problem overly simplistic for the Tianchi dataset.

A similar trend is observed in the Taobao dataset: Removing clicks reduced HR from 0.8515 to 0.7643, while removing “add-to-favorites” and “add-to-cart” behaviors caused only slight performance declines. Removing all auxiliary behaviors, however, led to a significant drop in performance, underscoring their importance for capturing user behavior.

In contrast, the Yelp and MovieLens datasets showed minimal differences in results when auxiliary behaviors were excluded. This is likely because the behaviors in these datasets are handcrafted and lack the same strong influence observed in the Tianchi and Taobao datasets.

4.6 Hyper-parameter tuning effect(RQ4)

The model's performance is influenced by several hyper-parameters, including sequence length, embedding size, and the classification task weight (Θ). To evaluate the model's

sensitivity to these hyper-parameters, we conducted experiments on the Tianchi and Taobao datasets. For the Tianchi dataset, as illustrated in Fig. 6, increasing the sequence length improved HR@10, reaching a peak of 0.787. The optimal embedding size for each item was 40, while a classification task weight of 0.3 yielded the best results for learning the behavior label. For the Taobao dataset, as shown in Fig. 7, increasing the sequence length to 200 resulted in a slight improvement in HR@10, peaking at 0.844. The best performance was achieved with an embedding size of 80 for each item and a classification task weight of 0.1 for learning the behavior label.

5 Hyper-parameter settings

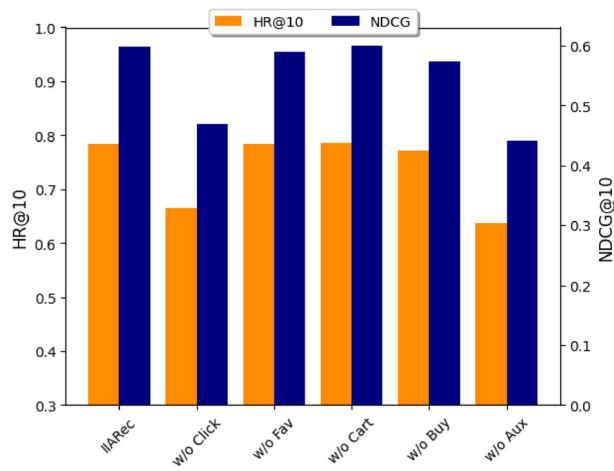
Our experiments were conducted using an RTX 2070 Super GPU and an Xeon Gold 6230 CPU with 256 GB of RAM. We implemented the models in TensorFlow.⁵ For hyper-parameter optimization, we performed a grid search across various ranges: latent embedding sizes [30–120], learning rates [0.001–0.00005], maximum sequence lengths [50–200], batch sizes [128, 256], and dropout rates [0.2–0.6]. The optimal parameters were found to vary by dataset, for **Taobao dataset**; dropout rate of 0.25, the maximum sequence length of 200, embedding size of 80, and learning rate of 0.0005. While for **Tianchi dataset**; dropout rate of 0.5, the maximum sequence length of 160, embedding size of 40, and learning rate of 0.0007. On the other hand, **MovieLens dataset**; dropout rate of 0.4, the maximum sequence length of 70, embedding size of 70, and learning rate of 0.0006. Finally, for **Yelp dataset**; dropout rate of 0.5, maximum sequence length of 100, embedding size of 50, and learning rate of 0.0003.

For all datasets, we found that a batch size of 128, with a single attention block and one head, yielded the best performance. We used the ReLU activation function across all model layers and in the feed-forward layer of the multi-head self-attention block, which led to improved results. Additionally, we tuned the parameter α_b within [0, 1] and the parameter β within [0, 2], finding that 1.1 for β provided the best results. Finally, we optimized the model using the Adam optimizer [10].

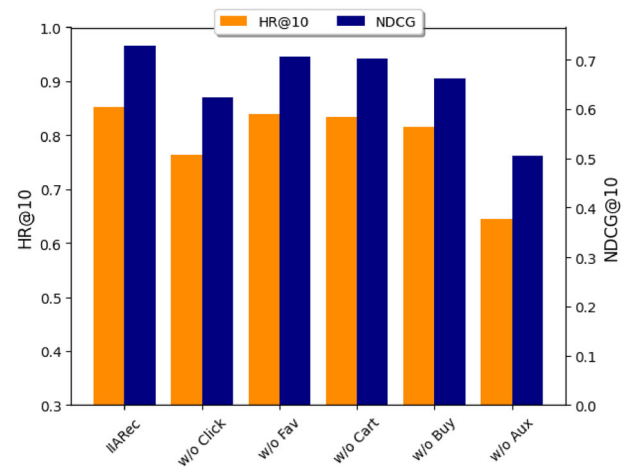
6 Conclusion

In this paper, we presented the IIARec model that captures the deep behavioral dependencies and sequential patterns in multi-behavioral data. The model encodes items of the same behavior using a behavior encoding component followed by a

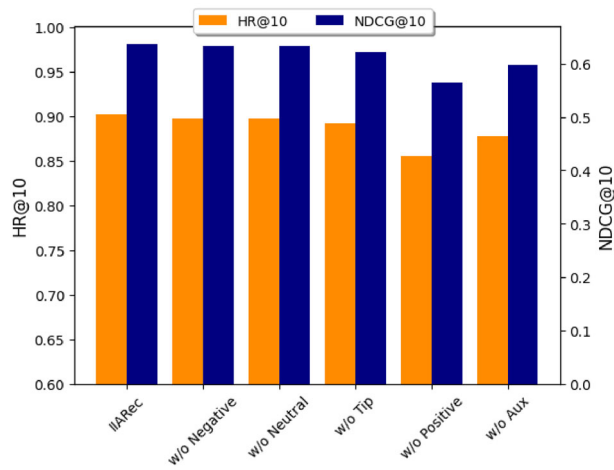
⁵ <https://www.tensorflow.org>.



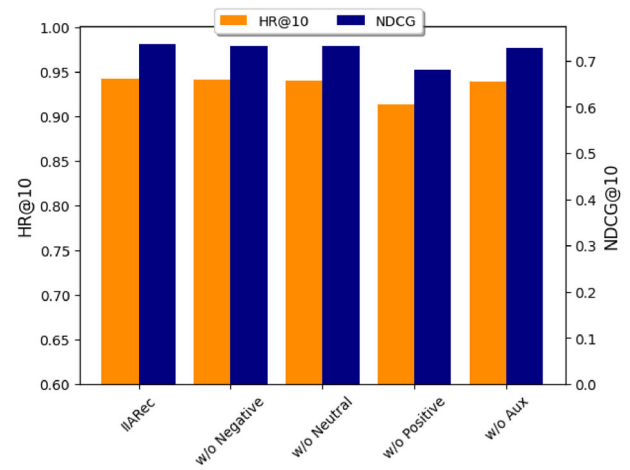
Tianchi Dataset



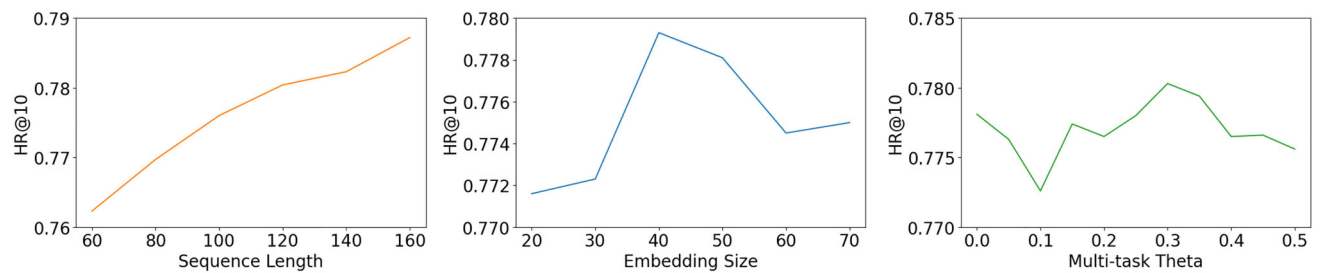
Taobao Dataset

Fig. 4 Effect of auxiliary behaviors on Tianchi and Taobao Datasets

Yelp Dataset



MovieLens Dataset

Fig. 5 Effect of auxiliary behaviors on Yelp and MovieLens Datasets**Fig. 6** Hyper-parameter tuning for Tianchi dataset

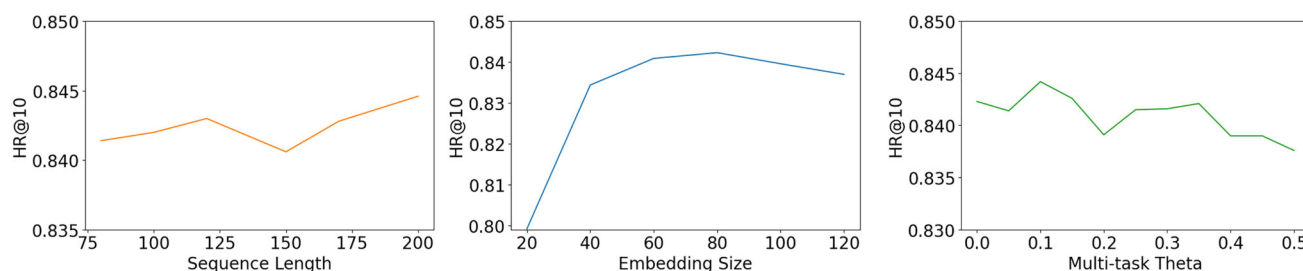


Fig. 7 Hyper-parameter tuning for Taobao dataset

sequence encoder to capture cross-behavior dependencies in the sequence. Furthermore, we empower our model by applying multitask learning of the behavior types. Experimental results on four multi-behavioral datasets show that our model outperforms state-of-the-art recommendation methods.

6.1 Limitations and future work

The presented work effectively captures the complex relationships between different user behaviors through an attention mechanism that models interactions both within the same behavior and across different behaviors. However, when the dataset contains a large number of behaviors, the computational cost of this approach can become prohibitively expensive. One potential solution to mitigate this issue is to group or combine similar behaviors before passing them through the behavior-specific attention layer, thereby reducing the computational complexity.

Additionally, the importance of behaviors can vary significantly from one user to another. For instance, some users may click on numerous items without making any purchases, indicating that click behavior is less significant for them compared to purchase behavior. On the other hand, other users may only click on items they intend to buy, making their click behavior highly predictive of their purchase intent. To address this variability, a possible solution is to make the behavior weights in the loss function learnable on a per user basis. While this approach could provide more personalized recommendations, it may also introduce significant computational overhead, particularly for datasets with a large number of users.

Funding Open Access funding enabled and organized by Projekt DEAL.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your

intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Bachlechner, T., Majumder, B.P., Mao, H., Cottrell, G., McAuley, J.: Rezero is all you need: Fast convergence at large depth. In: Uncertain. Artif. Intell. 1352–1361. PMLR (2021)
- Chen, Y., Liu, Z., Li, J., McAuley, J., Xiong, C.: Intent contrastive learning for sequential recommendation. In: proceedings of the ACM Web conference 2022. 2172–2182 (2022)
- Cheng, Z., Han, S., Liu, F., Zhu, L., Gao, Z., Peng, Y.: Multi-behavior recommendation with cascading graph convolution networks. In: Proceedings of the ACM Web Conference 2023. 1181–1189 (2023)
- Han, Y., Wang, H., Wang, K., Wu, L., Li, Z., Guo, W., Liu, Y., Lian, D., Chen, E.: Efficient noise-decoupling for multi-behavior sequential recommendation. In: Proceedings of the ACM Web Conference 2024. 3297–3306 (2024)
- He, X., Chua, T.S.: Neural factorization machines for sparse predictive analytics. In: Proceedings of the 40th International ACM SIGIR conference on Research and Development in Information Retrieval. 355–364 (2017)
- He, X., Liao, L., Zhang, H., Nie, L., Hu, X., Chua, T.S.: Neural collaborative filtering. In: Proceedings of the 26th international conference on world wide web. 173–182 (2017)
- Jannach, D., Ludewig, M.: When recurrent neural networks meet the neighborhood for session-based recommendation. In: Proceedings of the eleventh ACM conference on recommender systems. pp. 306–310 (2017)
- Jin, B., Gao, C., He, X., Jin, D., Li, Y.: Multi-behavior recommendation with graph convolutional networks. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. 659–668 (2020)
- Kang, W.C., McAuley, J.: Self-attentive sequential recommendation. In: 2018 IEEE international conference on data mining (ICDM). 197–206. IEEE (2018)
- Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. arXiv preprint [arXiv:1412.6980](https://arxiv.org/abs/1412.6980) (2014)
- Li, J., Wang, M., Li, J., Fu, J., Shen, X., Shang, J., McAuley, J.: Text is all you need: Learning language representations for sequential recommendation. arXiv preprint [arXiv:2305.13731](https://arxiv.org/abs/2305.13731) (2023)
- Li, J., Wang, Y., McAuley, J.: Time interval aware self-attention for sequential recommendation. In: Proceedings of the 13th international conference on web search and data mining. 322–330 (2020)
- Rashed, A., Elsayed, S., Schmidt-Thieme, L.: Context and attribute-aware sequential recommendation via cross-attention. In:

- Proceedings of the 16th ACM Conference on Recommender Systems. 71–80 (2022)
14. Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., Jiang, P.: Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In: Proceedings of the 28th ACM international conference on information and knowledge management. 1441–1450 (2019)
 15. Tang, J., Wang, K.: Personalized top-n sequential recommendation via convolutional sequence embedding. In: Proceedings of the eleventh ACM international conference on web search and data mining. 565–573 (2018)
 16. Wu, L., Li, S., Hsieh, C.J., Sharpnack, J.: Sse-pt: Sequential recommendation via personalized transformer. In: Fourteenth ACM Conference on Recommender Systems. 328–337 (2020)
 17. Xia, L., Huang, C., Xu, Y., Dai, P., Zhang, B., Bo, L.: Multi-plex behavioral relation learning for recommendation via memory augmented transformer network. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. 2397–2406 (2020)
 18. Xia, L., Huang, C., Xu, Y., Dai, P., Zhang, X., Yang, H., Pei, J., Bo, L.: Knowledge-enhanced hierarchical graph transformer network for multi-behavior recommendation. In: Proceedings of the AAAI Conference on Artificial Intelligence. 35, 4486–4493 (2021)
 19. Xia, L., Xu, Y., Huang, C., Dai, P., Bo, L.: Graph meta network for multi-behavior recommendation. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval. 757–766 (2021)
 20. Xuan, H., Liu, Y., Li, B., Yin, H.: Knowledge enhancement for contrastive multi-behavior recommendation. In: Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining. 195–203 (2023)
 21. Yang, Y., Huang, C., Xia, L., Liang, Y., Yu, Y., Li, C.: Multi-behavior hypergraph-enhanced transformer for sequential recommendation. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2263–2274 (2022)
 22. Yu, W., Bin, C., Liu, W., Chang, L.: Contrastive learning-based multi-behavior recommendation with semantic knowledge enhancement. In: 2023 IEEE International Conference on Data Mining (ICDM). 1511–1516. IEEE (2023)
 23. Yuan, E., Guo, W., He, Z., Guo, H., Liu, C., Tang, R.: Multi-behavior sequential transformer recommender. In: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval. 1642–1652 (2022)
 24. Zhou, K., Wang, H., Zhao, W.X., Zhu, Y., Wang, S., Zhang, F., Wang, Z., Wen, J.R.: S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In: Proceedings of the 29th ACM International Conference on Information & Knowledge Management. 1893–1902 (2020)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.